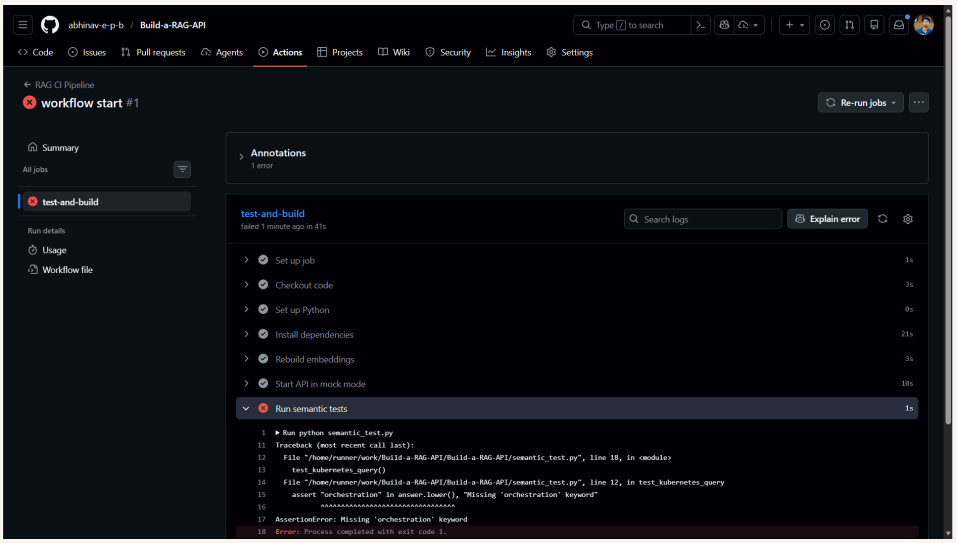
nextwork.org

Automate Testing with GitHub Actions

Abhinave P.B



Abhinave P.B
NextWork Student

Introducing Today's Project!

In this project, I will demonstrate how Github actions work. I'm doing this project to learn how automated testing works

Key services and concepts

Services I used were Github actions and git. Key concepts I learnt include version management and test automation.

Challenges and wins

This project took me approximately 3 hrs The most challenging part was workflow creation. It was most rewarding to do your own automation on your own

Why I did this project

I did this project because I wanted to learn amount github actions. One thing I'll apply from this is automates testing of my codes from now on

Abhinave P.B
NextWork Student

Setting Up Your RAG API

I’m setting up my RAG API by containerizing it with Docker so all dependencies, models, and configurations run consistently across environments. A RAG API retrieves information by embedding the query, performing vector similarity search, and using the retrieved documents as context for the LLM. This foundation is needed for CI/CD because Docker enables repeatable builds, automated testing, and reliable deployments without environment-related issues.

Local API verification

I tested my RAG API by sending a POST request to the /query endpoint using curl, passing a natural language question as a query parameter while the FastAPI server was running locally. The API responded with a generated answer from the Ollama LLM, which was created using the most relevant context retrieved from the Chroma vector database. This confirms that the Retrieval-Augmented Generation pipeline is working correctly: the API is successfully retrieving relevant documents from the vector store, injecting them into the prompt, and generating a meaningful response using the language model.

Abhinave P.B

NextWork Student


```
(venv) abhinave32@DESKTOP-32UV7P9:/mnt/c/Users/abhin/OneDrive/Desktop/Ironfleet/Archives/Build a RAG API$ ollama serve
Error: listen tcp 127.0.0.1:11434: bind: address already in use
(venv) abhinave32@DESKTOP-32UV7P9:/mnt/c/Users/abhin/OneDrive/Desktop/Ironfleet/Archives/Build a RAG API$ curl -X POST "http://127.0.0.1:8000/query?q=What%2
0is%20Kubernetes%3F"
{"answer":"Sure, here's the answer:\n\nKubernetes (K8s) is a container orchestration platform used to manage containers at scale."}(venv) abhinave32@DESKTOP
-32UV7P9:/mnt/c/Users/abhin/OneDrive/Desktop/Ironfleet/Archives/Build a RAG API$
```

Abhinave P.B
NextWork Student

Initializing Git and Pushing to GitHub

I’m initializing Git by creating a new local repository using git init, which sets up Git to start tracking the project files and their history. Git tracks changes by recording snapshots of files through commits, allowing me to see what changed, when it changed, and to roll back to previous versions if needed. Version control enables CI/CD to automatically build, test, and deploy the application whenever changes are pushed to the repository, ensuring faster development, consistency, and reduced risk of errors in production.

Git initialization and first commit

I initialized Git by "git init" command Then, I staged and committed using "git commit". The .gitignore file helps by not tracking files which are not needed for other users

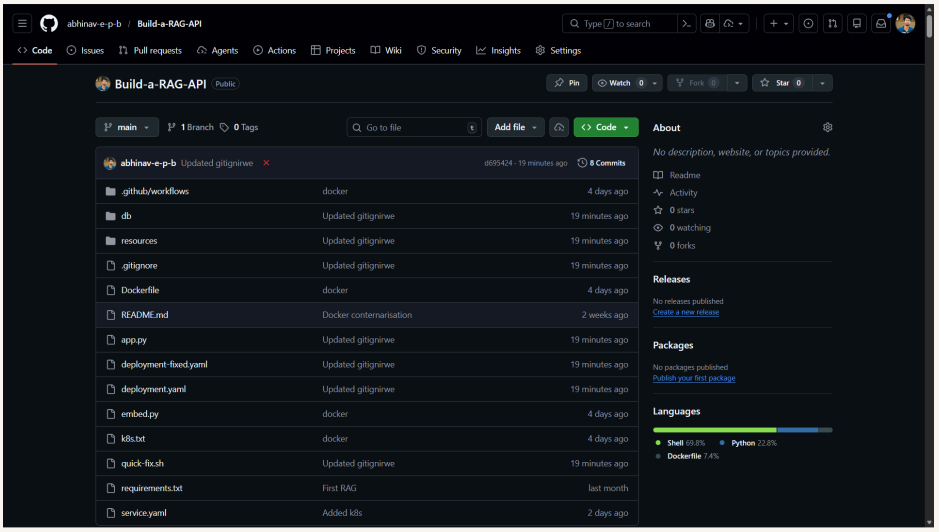
Pushing to GitHub for CI/CD

Pushing to GitHub means uploading your files to Github account so that others can see your code. This enables CI/CD because github actions

Abhinave P.B

NextWork Student

nextwork.org



Abhinave P.B
NextWork Student

Creating Semantic Tests

I’m creating semantic tests that verify the meaning and relevance of responses produced by my RAG API, rather than just checking for syntactically correct outputs. Unlike unit tests, which validate individual code logic and deterministic behavior, semantic tests evaluate whether the API understands a query and retrieves contextually accurate information from the knowledge base. These tests ensure response quality, consistency, and reliability by confirming that answers remain correct even when the wording of questions changes or when the knowledge base is modified.

Non-deterministic output observation

When I ran the query multiple times, I noticed that the results were inconsistent—sometimes returning the expected data, other times failing or returning incomplete information. This is a problem because inconsistent query results can break automated pipelines and lead to unreliable builds or deployments. For CI/CD to work reliably, we need deterministic behavior from our queries so that each pipeline run produces predictable outcomes, ensuring that tests, deployments, and validations can be trusted without manual intervention.

Abhinave P.B
NextWork Student

Adding Mock LLM Mode

I'm adding mock LLM mode to the API. This solves the non-determinism problem by returning predefined, consistent responses instead of relying on the real LLM, which may produce slightly different outputs on each run. Reliable testing requires deterministic behavior, so that each test run produces the same results every time, allowing CI/CD pipelines and automated tests to validate functionality confidently.

Mock LLM mode for CI testing

Mock LLM mode is a testing feature used in applications that rely on large language models (LLMs), where instead of actually calling the LLM API, the system returns the retrieved text directly. This makes tests faster, deterministic, and predictable, which is essential because real LLM calls can be slow, expensive, and produce variable outputs depending on network conditions or the model’s responses. Without mock mode, tests could fail intermittently or consume unnecessary API credits, making automated CI/CD pipelines unreliable and costly. By using mock mode, developers ensure that tests run quickly, consistently, and cost-effectively, while also simplifying debugging since the output is predictable.

Abhinave P.B
NextWork Student

Creating GitHub Actions Workflow

I'm creating a GitHub Actions workflow file that automates testing for my project. The workflow runs predefined tests automatically whenever I push code to the repository, ensuring that any changes are validated immediately. By doing this, I can catch errors early, maintain code quality, and streamline the development process without manually running tests each time.

Workflow automation and CI testing

I created the workflow file in my project’s .github/workflows directory and pushed it to the repository using Git. Once on GitHub, the workflow will automatically run whenever I push code or trigger the configured events, executing the defined steps such as running tests, building the project, or performing other automated tasks.

Abhinave P.B
NextWork Student

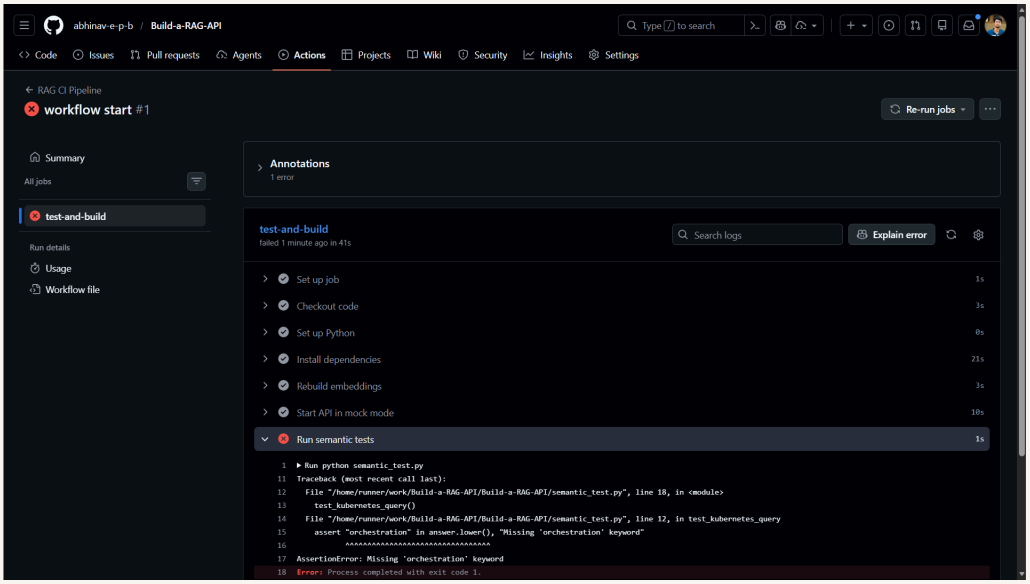
Testing Data Quality

I'm triggering the CI workflow by making changes to the files specified in ci.yml file and pushing to github.The workflow will test my code automatically. I expect it to fail because of k8s.txt text i have given

Data quality and CI protection

The missing keyword was "orchestration," which caused the semantic test to fail. Without CI, this degraded content would have been deployed to the knowledge base, potentially leading to incomplete or misleading information in the system’s responses. By catching it automatically, CI ensured that only high-quality, accurate content was included, protecting the integrity and reliability of the knowledge base.

Abhinave P.B
NextWork Student



Abhinave P.B
NextWork Student

Scaling with Multiple Documents

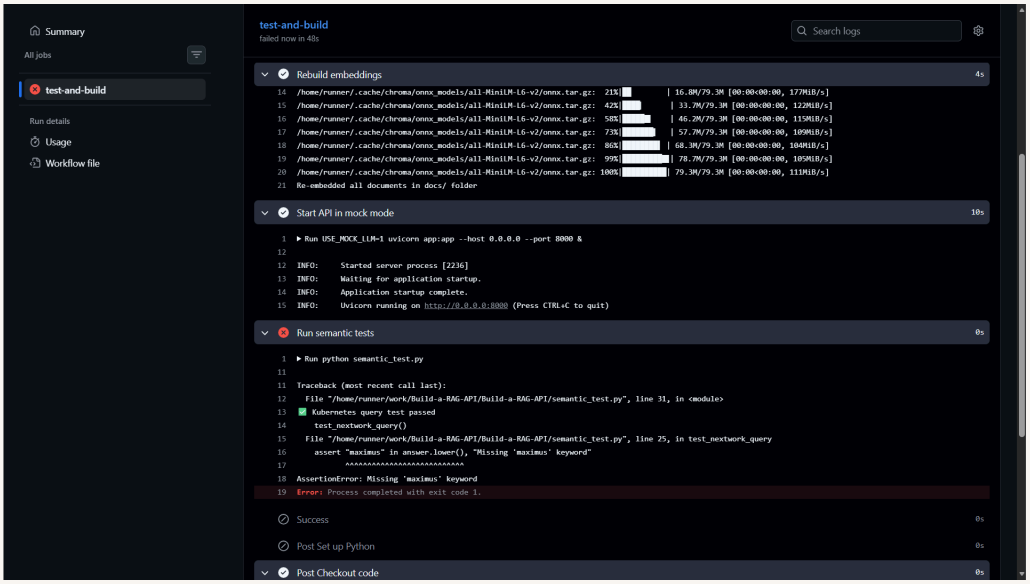
I'm restructuring the project to handle multiple knowledge documents. The new folder structure supports storing each topic or module in a separate file within a central knowledge/ directory, allowing the RAG system to automatically ingest all documents. This approach scales better because adding new content no longer requires code changes, and CI can automatically test every file for quality and semantic correctness, ensuring the knowledge base remains accurate and reliable even as it grows.

Docs folder structure and CI scaling

The docs folder organizes files by individual knowledge documents rather than a single hardcoded source. The embed_docs.py script handles iterating over all files in the folder, chunking them, generating embeddings, and storing them in the vector database. CI validated all documents and found that semantic queries pass consistently across the entire knowledge base, catching missing or degraded content early. This structure supports growth by allowing new knowledge to be added, removed, or updated simply by modifying files in the docs directory without changing application code.

Abhinave P.B
NextWork Student

nextwork.org



The place to learn & showcase your skills

Check out nextwork.org for more projects